

Zaraz Store

☐ *Table of Contents*

- 1. Problem Statement and Objectives***
- 2. System Design***
- 3. Code Implementation***
- 4. Test Cases***
- 5. Challenges and Future Improvements***

1. Problem Statement and Objectives

- Problem Statement:

The Zaraz Store project addresses the need for an online e-commerce platform where users can browse products, add them to the cart, and proceed to checkout using different payment methods. It provides a user-friendly interface with the option to choose from various payment options like Bkash, Card, and Cash on Delivery.

- Objectives:

The primary goals of the Zaraz Store project are:

1. To provide a seamless shopping experience for users, allowing them to easily browse products and add them to their cart.
2. To allow users to enter their personal information (name, address, phone number) and select their preferred payment method during checkout.
3. To generate an invoice that includes product details, payment method, and other relevant user information.
4. To enhance the user experience with an organized cart and checkout flow, as well as providing clear product and billing information.

2. System Design

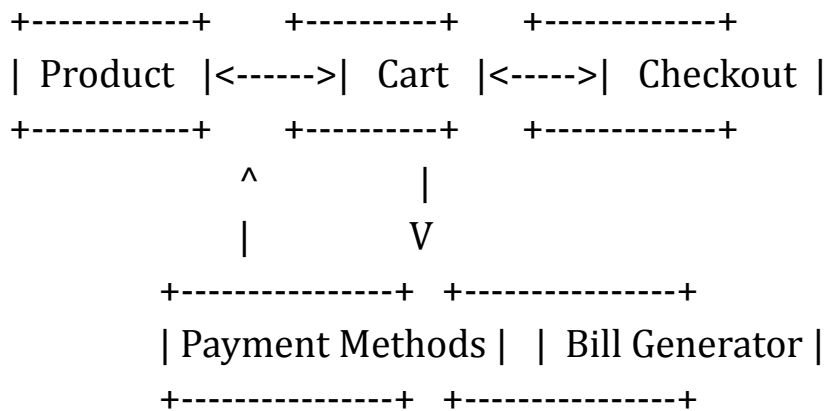
- Overview:

The Zaraz Store system follows a modular design using the Model-View-Controller (MVC) architecture. The project is built with ASP.NET using C#. The system consists of several key components such as:

- Product Display: Products are shown with name, description, and price.
- Cart: Users can add products to their cart, view, update, or remove them.
- Checkout: Users enter personal information and select payment methods.
- Payment Modules: Each payment method (Bkash, Card, Cash on Delivery) has a separate page for processing.
- Bill Generator: After payment is processed, the system generates a bill with detailed transaction information.

- Diagrams:

The system architecture can be represented as follows:



3. Code Implementation

- Overview:

The codebase for Zaraz Store follows the MVC architectural pattern, separating data models, business logic, and UI components. Each page (Index, Cart, Checkout, Payment, Bill) is handled by its respective controller that processes the HTTP requests and generates the appropriate views.

- Key Modules:

1. Product Module:

Displays products with name, description, and price. The controller fetches product data and renders it on the Index page.

2. Cart Module:

Adds, views, updates, and removes products in the cart. The cart data is stored in session variables.

3. Checkout Module:

Collects user details like name, address, and phone number. It validates the data before submitting it to the payment methods.

4. Payment Module:

Each payment method (Bkash, Card, Cash) is implemented as a separate form.

5. Bill Generator:

After payment, a bill page is generated displaying user information and product details.

- Code Snippets:

- **Example for adding and removing a product to the cart:**

```
public IActionResult AddToCart(int productId)
{
    var product = GetProductById(productId);

    if (product == null) return RedirectToAction("Index");

    var cart = HttpContext.Session.GetObjectFromJson<List<CartItem>>("Cart") ??
new List<CartItem>();

    var existingItem = cart.FirstOrDefault(x => x.ProductId == productId);

    if (existingItem != null)
    {
        existingItem.Quantity++;
    }
    else
    {
        cart.Add(new CartItem(
            productId, product.Name, product.Description, product.ImageUrl,
            "Default Category", 1, product.Price));
    }

    HttpContext.Session.SetObjectAsJson("Cart", cart);

    return RedirectToAction("Cart");
}
```

```

// Remove a product from the cart

[HttpPost]

public IActionResult RemoveFromCart(int productId)

{

    var cart = HttpContext.Session.GetObjectFromJson<List<CartItem>>("Cart") ??
new List<CartItem>();

    var itemToRemove = cart.FirstOrDefault(p => p.ProductId == productId);

    if (itemToRemove != null)

    {

        cart.Remove(itemToRemove);

        HttpContext.Session.SetObjectAsJson("Cart", cart);

    }

    return RedirectToAction("Cart");

}

```

4. Test Cases

- Overview:

The Zaraz Store project was tested manually by interacting with the forms, adding products to the cart, completing the checkout process, and simulating different payment methods.

- CRUD Operations:

1. Create: Adding products to the cart and submitting the checkout form.

ZARAZ STORE

Your favorite online store for the best deals, fast delivery, and top brands!

Go to Cart

Welcome to ZARAZ!

Discover top deals, popular brands, and fast delivery on thousands of items.

Shop Now

Featured Products

Product1

Product1

Laptop

\$10.99

Add to Cart

Product2

Product2

Mouse

\$20.99

Add to Cart

Product3

Product3

Cable

\$30.99

Add to Cart

Product4

Product4

Ram

\$15.99

Add to Cart

Product5

Product5

CPU

\$25.99

Product6

Product6

Monitor

\$35.99

Product7

Product7

Apple iphone

\$40.99

Product8

Product8

Iphone 15

\$50.99

Your Cart

Product	Quantity	Price (each)	Total	Action
Product1	1 Update	\$10.99	10.99.ToString("C")	Remove
Total:			\$10.99	

Continue Shopping

Checkout

© 2025 ECommerceApp. All rights reserved.

2. ****Read****: Displaying the cart and user information on the checkout page.

ECommerceApp

[Home](#)[Products](#)[Cart](#)

Checkout

Full Name

Nafees Al Mahadi

Phone Number

01316305345

Format: 01XXXXXXXX

Shipping Address

Mirpur-2,Dhaka-1216.

Payment Method

☐ Bkash

☒ Card (Debit/Credit)

☐ Cash on Delivery

Submit Order

© 2025 ECommerceApp. All rights reserved.

3. ****Update****: Updating the quantity of products in the cart.

Your Cart

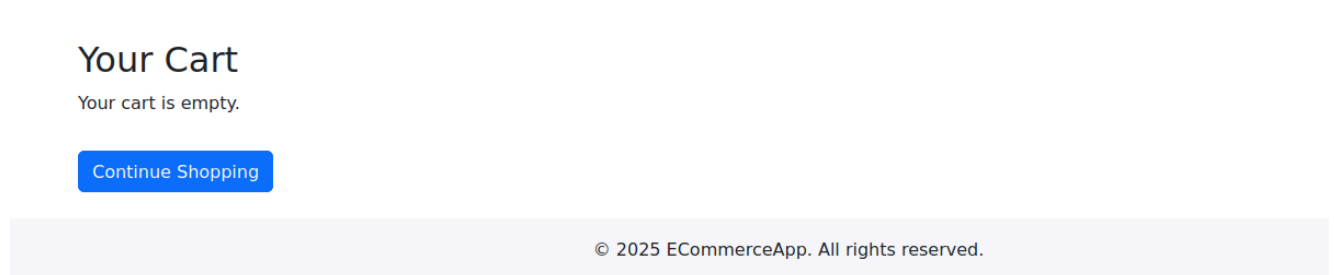
Product	Quantity	Price (each)	Total	Action
Product2	3 Update	\$20.99	20.99.ToString("C")	Remove
Total:			\$20.99	

Continue Shopping

Checkout

© 2025 ECommerceApp. All rights reserved.

4. ****Delete****: Removing products from the cart.



5. Challenges and Future Improvements

- Challenges Encountered:

1. Session Management:

Handling cart data through session variables had issues initially, as some products were not updating correctly. This was resolved by carefully managing session data and adding logging.

2. Multiple Payment Methods:

Implementing separate forms for each payment method caused routing conflicts. This was solved by defining clear actions and route paths for each payment option.

3. Bill Generation:

Calculating totals for the products was complex, as prices had to be multiplied by the quantity dynamically. Using proper loops solved this issue.

4. User Interface:

Styling the application to match a professional design required a lot of CSS work, and ensuring compatibility with all browsers was a challenge.

- Future Improvements:

1. User Authentication:

Adding user login and registration functionality will allow users to track orders.

2. Real Payment Integration:

Integrating real payment gateways (Stripe, SSLCOMMERZ) will allow actual payments rather than mock payments.

3. Product Admin Panel:

An admin panel where the store owner can add, update, and delete products easily.

4. Responsive Design:

Implementing a responsive design framework like Bootstrap to improve mobile compatibility.

5. PDF Bill Generation:

Allowing users to download the bill in PDF format for their records.

Appendix

- **References:**

- **W3 School**
- **.Net ASP**
- **Chat GPT**

- **Additional Resources:**

–N/A.

Thankyou!